

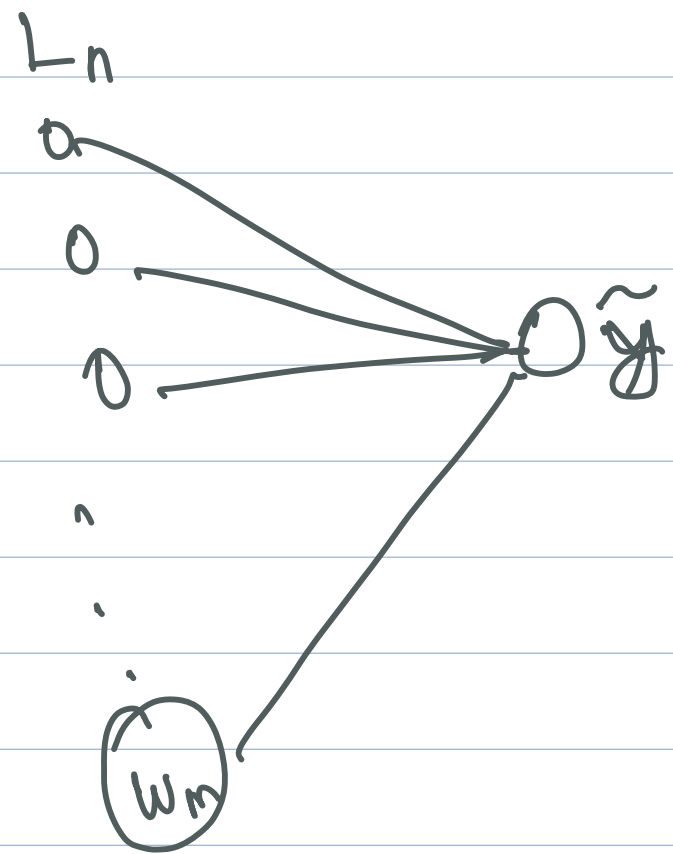
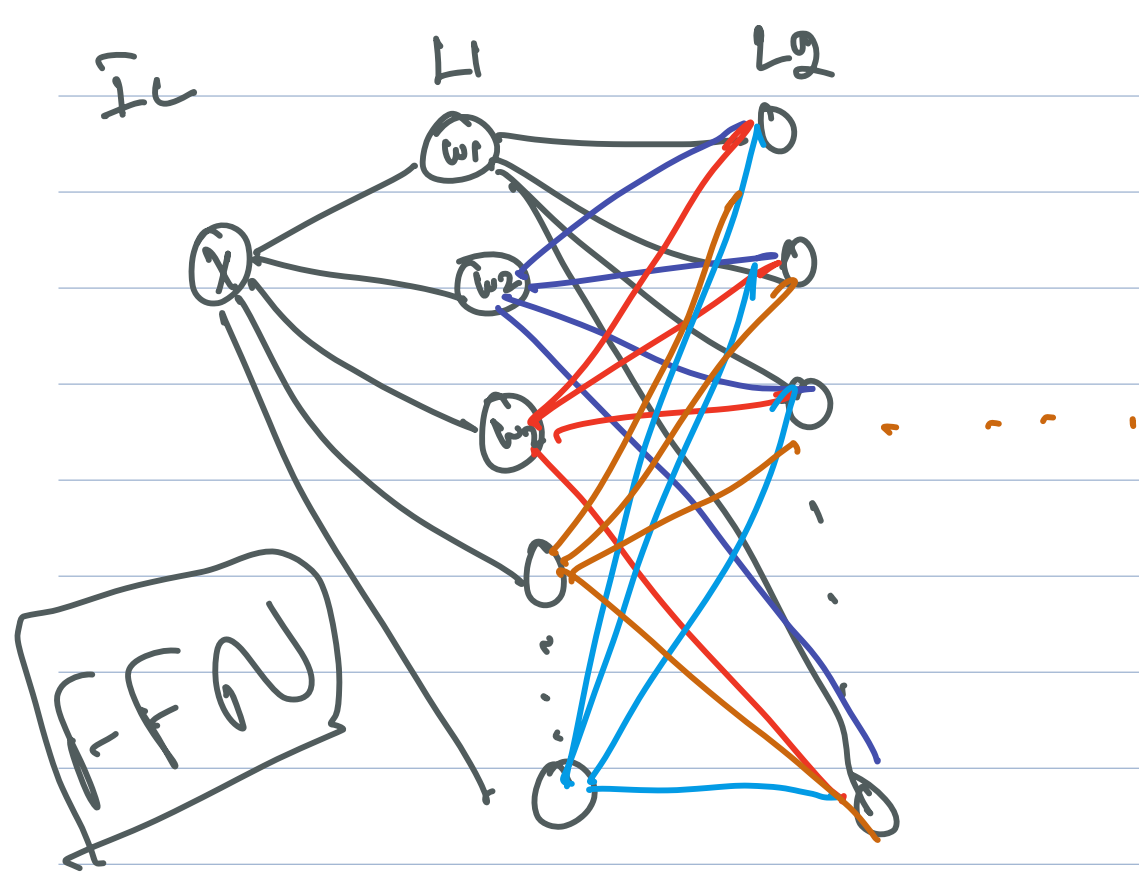
MODULE 10

Distributed Training

Data · Model · Pipeline · Hybrid Parallelism at Scale

Prof. Sashikumaar Ganesan · IISc Bangalore · AhamX Bodhi

Start @
11 am



$$L(w) = \frac{1}{2} \| \hat{y} - y^{\text{exact}} \|^2$$

where

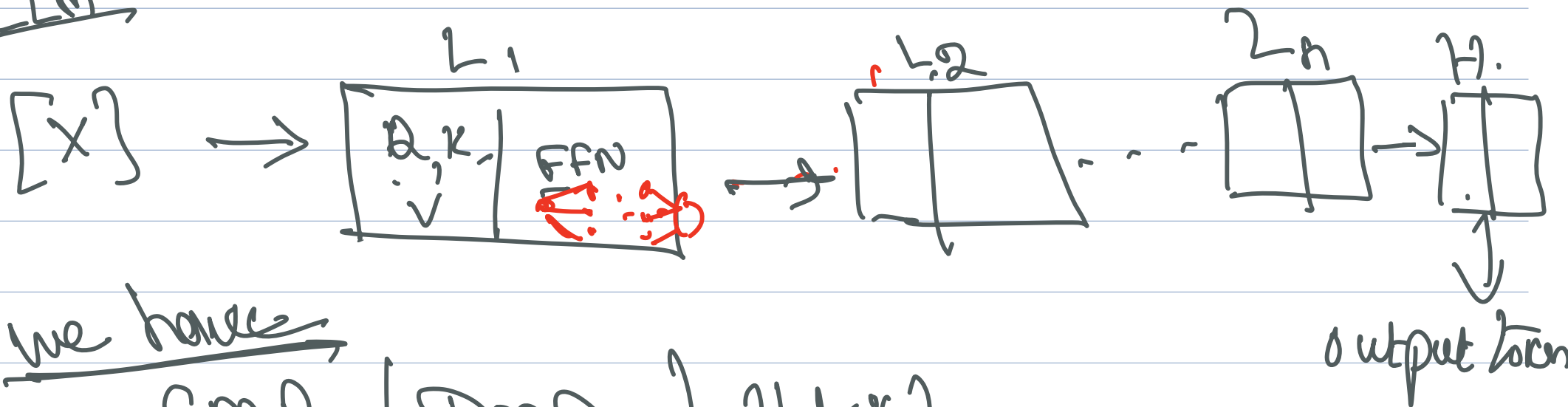
$$w = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}$$

Back Prop:

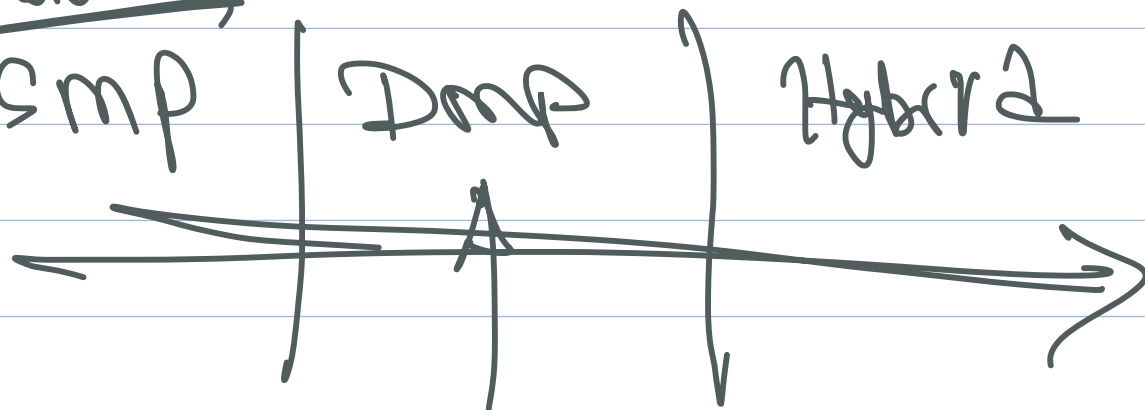
$$W^{K+1} = W^K - \eta \nabla_W L$$

calculate gradient using auto diff

LLM



we have
Smp



Overview



TH-000006

Distributed Fundamentals

Why single GPU fails, interconnects, NCCL



TH-000007

Data Parallelism

DDP, all-reduce, ring-allreduce, ZeRO



TH-000008

Model Parallelism

Tensor parallelism, layer splitting



TH-000009

Pipeline Parallelism

GPipe, microbatches, bubble overhead



CD-000002

DT Implementation

PyTorch DDP · FSDP · DeepSpeed · 30 min lab

LEARNING OBJECTIVES

Explain

Why a single GPU is insufficient for large model training and what breaks first

Distinguish

Data parallelism, model parallelism, and pipeline parallelism — and when to use each

Trace

The ring-allreduce algorithm step by step and calculate its communication volume

Derive

Pipeline bubble overhead formula and evaluate the trade-off with microbatch count

Apply

ZeRO optimizer stages to reduce memory footprint without reducing model quality

Implement

Multi-GPU training with PyTorch DDP and FSDP in working Python code

§1

Why Distributed Training?

The memory wall · GPU limits · When single-GPU breaks

The Case for Distributed Training

Single GPU hits hard ceilings long before frontier scale

5 A100

175B

GPT-3 parameters

Requires ~350 GB FP16 weights alone — far beyond any single GPU

80 GB

A100 max VRAM

Training a 7B model needs ~70 GB just for weights+gradients+optimizer

~2x

Data parallel speedup per GPU

Near-linear scaling to 64 GPUs with DDP; beyond that, communication dominates

Model	Params	Inference (FP16)	Training (FP32 Adam)	GPUs Needed
GPT-2	1.5B	3 GB	24 GB	1x A100
LLaMA-3 8B	8B	16 GB	~96 GB	2x A100 80G
LLaMA-3 70B	70B	140 GB	~840 GB	12x A100 80G
GPT-4 (est.)	~1.8T	~3.6 TB	~20 TB	100s of H100s

DGX-System





GPU Interconnects & Communication

Bandwidth determines how fast gradients can be exchanged

Interconnect	Bandwidth	Scope	Use case
PCIe Gen4	64 GB/s	Within node	Budget multi-GPU, CPU↔GPU
NVLink 4.0	900 GB/s	Within node	High-bandwidth GPU↔GPU (A100/H100)
NVSwitch	7.2 TB/s	DGX pod (8 GPUs)	Full bisection bandwidth within server
InfiniBand HDR	200 Gb/s	Multi-node	GPU cluster inter-node (RDMA)
Ethernet (RoCE)	100 Gb/s	Multi-node	Cost-effective cluster networking

NCCL (NVIDIA Collective Communications Library)

- Automatically selects fastest path (NVLink > PCIe)
- Implements AllReduce, Broadcast, Gather, Scatter
- Used by PyTorch DDP, FSDP, DeepSpeed

AllReduce

Sum gradients across all GPUs → all GPUs get identical result. Core of DDP.

Broadcast

Send model weights from rank-0 to all GPUs at init.

AllGather

Collect sharded weights from all GPUs (used in FSDP/ZeRO-3).

ReduceScatter

Reduce then scatter — each GPU gets a slice of the result (ZeRO-1/2).

TPUs
↳ maxText

MPI implementation

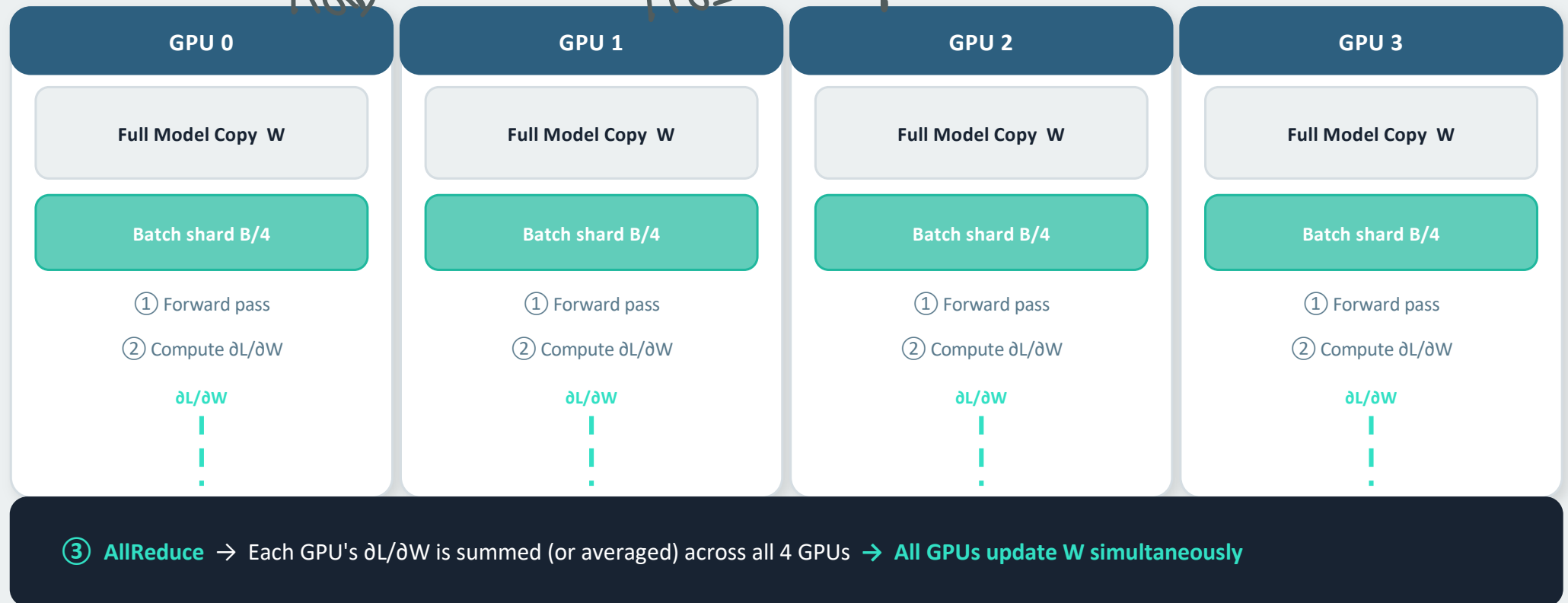
§2

Data Parallelism

Same model on every GPU · Different data batches · Gradient sync

Data Parallelism — How It Works

Each GPU holds a full model copy; gradients are averaged after backward pass





Ring-AllReduce — Traced Example

4 GPUs, gradient vector split into 4 chunks

Phase 1: Reduce-Scatter (N-1 steps, each GPU sends to its right neighbour)

GPU 0	[A0 B0 C0 D0]	<i>Owns: chunk 0 $\rightarrow$ $\text{sum}(A0, B0, C0, D0)$</i>
GPU 1	[A1 B1 C1 D1]	<i>Owns: chunk 1 $\rightarrow$ $\text{sum}(B1, C1, D1, A1)$</i>
GPU 2	[A2 B2 C2 D2]	<i>Owns: chunk 2 $\rightarrow$ $\text{sum}(C2, D2, A2, B2)$</i>
GPU 3	[A3 B3 C3 D3]	<i>Owns: chunk 3 $\rightarrow$ $\text{sum}(D3, A3, B3, C3)$</i>

Phase 2: All-Gather (N-1 more steps — propagate reduced chunks to everyone)

Result: Every GPU ends with the same fully-reduced gradient: [A0+A1+A2+A3 | B0+B1+B2+B3 | C0+C1+C2+C3 | D0+D1+D2+D3]

Communication cost: $2(N-1)/N \times |\text{gradient}| \approx 2 \times \text{gradient size for large } N$ — bandwidth-optimal!

DDP vs. Parameter Server

Two architectures for gradient synchronisation



Distributed Data Parallel (DDP)

- All-to-all communication (ring-allreduce)
- Every GPU is symmetric — no bottleneck node
- Gradients averaged across all workers simultaneously
- Best for: dense models, NVLink / InfiniBand clusters
- Used by: PyTorch DDP, Horovod
- Bandwidth efficiency: $O(2(N-1)/N)$ — near-optimal
- Fault tolerance: poor (one failure stops training)



Parameter Server (PS)

- Central PS node aggregates all gradients
- Workers send gradients to PS, receive updated weights
- Supports asynchronous updates (staleness tolerance)
- Best for: sparse models (embeddings), heterogeneous workers
- Used by: TensorFlow v1, MXNet, original DistBelief
- Bandwidth: PS is bottleneck — $O(N)$ traffic at server
- Fault tolerance: better (workers can be replaced)

§3

Model Parallelism

When the model is too big for one GPU · Tensor · Pipeline

Model Parallelism — Overview

Split the model itself across GPUs when it doesn't fit

When to use: A single model layer (e.g. a transformer attention matrix for 175B params) exceeds single-GPU VRAM. Data parallelism alone is insufficient. We must partition the model itself.

Naive Layer Split

Different layers on different GPUs

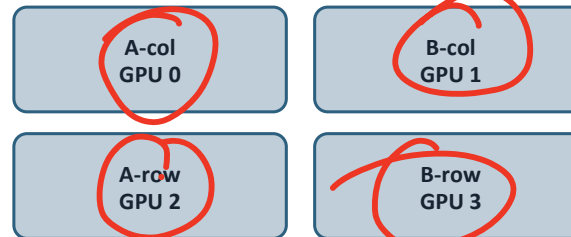


✓ Simple to implement

⚠ GPUs idle while waiting (pipeline bubble)
Low GPU utilisation

Tensor Parallelism

Each layer split across GPUs

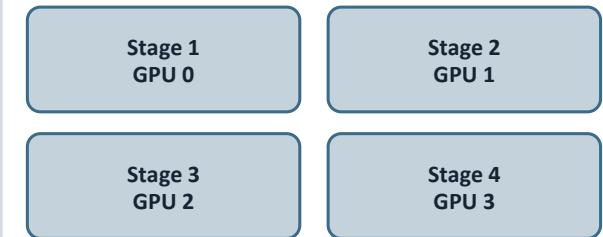


✓ All GPUs active every step
Megatron-LM standard

⚠ Heavy intra-layer comms
Needs NVLink bandwidth

Pipeline Parallelism

Microbatches flow through stages



✓ Lower inter-node bandwidth
Scales to many nodes

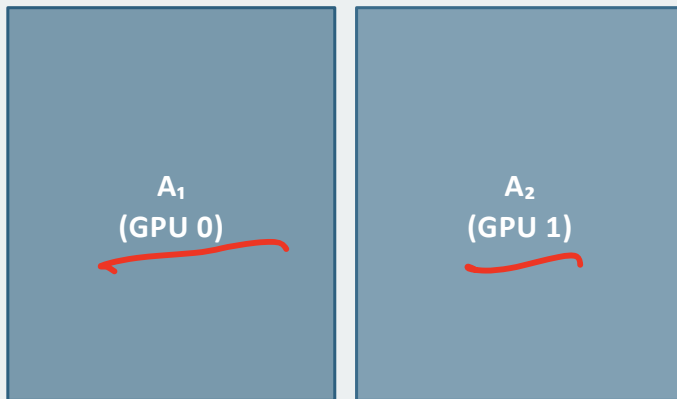
⚠ Bubble overhead (idle time)
Complex scheduling

Tensor Parallelism (Megatron-LM Style)

Split weight matrices column-wise then row-wise across GPUs

Column-Parallel Linear (MLP Layer 1)

Input X (shared across GPUs)



$W = [A_1 \mid A_2]$ (column split)

Output: $Y_1 = XA_1$ (GPU 0), $Y_2 = XA_2$ (GPU 1)

After Row-Parallel layer: AllReduce $\rightarrow Y = Y_1 + Y_2$

Key Properties

Synchronisation:

Only 1 AllReduce per transformer layer (vs DDP: 1 per step)

Memory saving:

Each GPU holds $1/N$ of each weight matrix

Bandwidth need:

Requires NVLink — 900 GB/s vs PCIe 64 GB/s

Activation memory:

Each GPU holds $1/N$ of intermediate activations

Standard impl.:

Megatron-LM, DeepSpeed Megatron, NeMo

Practical limit:

Typically $N \leq 8$ (one node) due to latency

Pipeline Parallelism (GPipe)

Microbatches keep all GPU stages busy simultaneously

Naive layer split — massive idle time (pipeline bubble):



GPipe — 4 microbatches (m=4): all stages busy most of the time



Bubble overhead: $(p-1)/(m+p-1)$ where p =pipeline stages, m =microbatches. More microbatches $\rightarrow$ smaller bubble.

Pipeline Bubble — Calculation

Quantify the overhead and choose the right microbatch count

$$\text{Bubble fraction} = (p - 1) / (m + p - 1)$$

p = number of pipeline stages (= number of GPU groups) m = number of microbatches per batch

p (stages)	m=1	m=4	m=8	m=16	m=32
p=2	50.0%	20.0%	11.1%	5.9%	3.0%
p=4	75.0%	42.9%	27.3%	15.8%	8.6%
p=8	87.5%	63.6%	46.7%	30.4%	17.9%
p=16	93.8%	78.9%	65.2%	48.4%	31.9%

Red = >50% idle. Yellow = 20–50%. Green = <10% — acceptable.

Rule of thumb: Set $m \geq 4 \times p$ to keep bubble below 20%. For $p=8$ stages, use $m \geq 32$ microbatches. Each microbatch must fit in GPU memory.

§4

Hybrid & Advanced Strategies

3D Parallelism · ZeRO · Mixed Precision · Gradient Checkpointing

3D Parallelism — Hybrid Strategy

Combine all three for frontier-scale training

The Parallelism Cube

Data Parallel (DP)

Same model, different data
Axis: Replicas

Tensor Parallel (TP)

Same layer, different tensors
Axis: Within a node

Pipeline Parallel (PP)

Different layers, microbatches
Axis: Across nodes

$$\text{Total GPUs} = \text{DP} \times \text{TP} \times \text{PP}$$

e.g. 2048 GPUs = 16 DP × 8 TP × 16 PP (Megatron-LM GPT-3)

Production Configurations

Model	DP	TP	PP	GPUs	Framework
GPT-3 175B	64	8	4	2048 A100s	Megatron-LM
LLaMA-3 70B	8	4	2	64 A100s	Meta internal
LLaMA-3 8B	8	2	1	16 A100s	Single node TP
GPT-2 1.5B	4	1	1	4 GPUs	DDP only fine

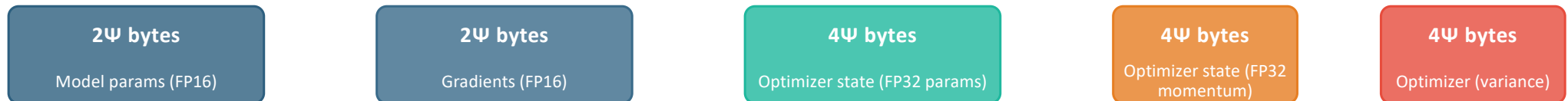
Decision guide: Set TP = 8 (one full NVLink node). Then stack PP across nodes. Use DP to scale to total GPU budget. Always saturate TP first (cheapest communication).



ZeRO Optimizer — Zero Redundancy

Shard optimizer states, gradients, and parameters across DP workers

Memory breakdown for a model with Ψ parameters (Adam optimizer, mixed precision):

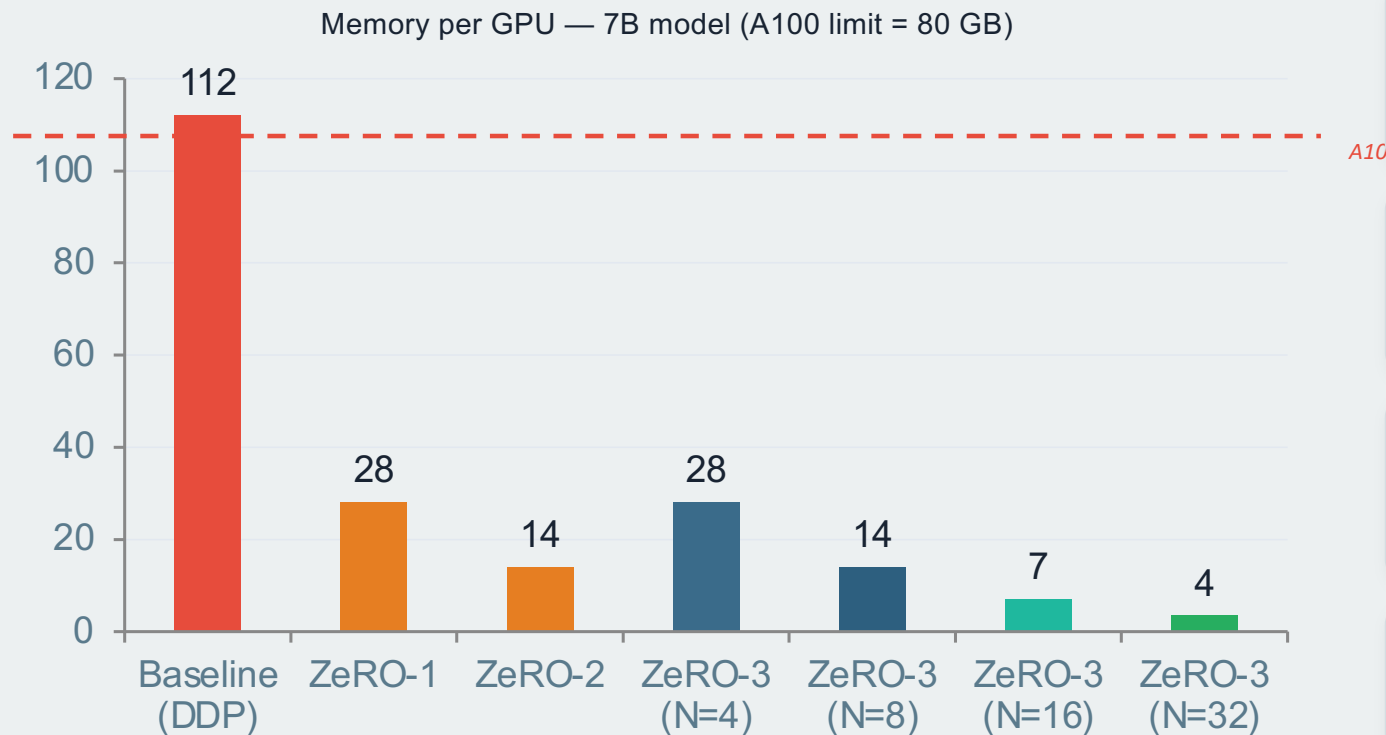


Total: 16 Ψ bytes per GPU (= 112 GB for a 7B model!)

ZeRO Stage	What is sharded	Memory / GPU	Communication overhead	Equivalent to
Baseline (DDP)	Nothing sharded	16 Ψ / 1 bytes	AllReduce (2 Ψ)	Standard DDP
ZeRO-1	Optimizer states	4 Ψ bytes (-75%)	AllReduce (2 Ψ)	DeepSpeed Stage 1
ZeRO-2	Optimizer + Gradients	2 Ψ bytes (-87%)	AllReduce (2 Ψ)	DeepSpeed Stage 2
ZeRO-3	All: params+grad+optim	16 Ψ /N bytes (-1/N)	AllGather+ReduceScatter (3 Ψ)	FSDP / DS Stage 3
ZeRO-Infinity	Offload to CPU/NVMe	$\sim\Psi$ bytes	CPU $\leftrightarrow$ GPU transfers	DeepSpeed Infinity

ZeRO Memory Savings vs. GPU Count

Memory per GPU decreases linearly with number of data-parallel workers



ZeRO-1

Use first — minimal overhead, ~4× memory reduction. Always enable with DeepSpeed.

ZeRO-2

Add gradient sharding. Good for 13B models on 8×A100s.

ZeRO-3 / FSDP

Full sharding. Enables 70B+ on commodity GPU clusters. Extra AllGather cost.

ZeRO-Infinity

Offload to CPU RAM / NVMe. Enables trillion-parameter training. Slow.

Mixed Precision Training

Store weights in FP16/BF16 to halve memory and 2× throughput

FP32

32-bit · 4 bytes/param

Range: $\pm 3.4 \times 10^{38}$ | ~7 decimal digits

Legacy; optimizer master copy

FP16

16-bit · 2 bytes/param

Range: $\pm 65,504$ | ~3 decimal digits

Activations, gradients; needs loss scaling

BF16

16-bit · 2 bytes/param

Range: $\pm 3.4 \times 10^{38}$ | ~2 decimal digits

Training on A100/H100; no loss scaling needed

INT8

8-bit · 1 byte/param

Range: -128 to 127 | Integer only

Inference only (LLM.int8, AWQ)

PyTorch AMP (autocast)

```
from torch.cuda.amp import autocast,
    GradScaler

scaler = GradScaler() # for FP16

for batch in loader:
    with autocast(dtype=torch.bfloat16):
        output = model(batch) # BF16
        loss = criterion(output)
    # BF16 needs NO loss scaling
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()
```

BF16 preferred on A100/H100. FP16 needs loss scaling. Both give ~2× memory + throughput vs FP32.

Gradient Checkpointing

Trade compute for memory — recompute activations instead of storing them

Standard Backprop

- Forward pass: store ALL intermediate activations
- Memory: $O(\text{layers})$ — grows linearly with model depth
- For GPT-3: ~375 GB of activations alone
- GPU OOM on large models

$O(L)$ memory

Gradient Checkpointing

- Forward pass: store only activations at checkpoints
- Backward pass: recompute activations between checkpoints
- Memory: $O(\sqrt{L})$ — square-root reduction
- Cost: ~33% extra compute (one extra forward pass)

$O(\sqrt{L})$ memory

All Strategies — Master Comparison

Choose your parallelism based on model size and hardware topology

Strategy	Memory reduction	Compute overhead	Communication	Best for	Framework
Data Parallel (DDP)	None (N copies of model)	None	AllReduce (2Ψ per step)	<70B, single-node multi-GPU	PyTorch DDP
ZeRO-1/2	4× / 8×	~1%	AllReduce	General purpose	DeepSpeed
ZeRO-3 / FSDP	N× (full sharding)	2× AllGather	AllGather + ReduceScatter	70B+ on many nodes	FSDP / DeepSpeed 3
Tensor Parallel	1/N per weight matrix	Minimal	AllReduce per layer	Large layers, 1 node (NVLink)	Megatron-LM
Pipeline Parallel	1/p per stage	Bubble $(p-1)/(m+p-1)$	Point-to-point (activations)	Multi-node, large models	GPipe, Megatron-LM
3D (DP+TP+PP)	Full combination	Small overhead	All of the above	Frontier (100B+ models)	Megatron-DeepSpeed
Mixed Precision	2× (FP16/BF16)	None (faster!)	Unchanged	All training	torch.cuda.amp
Grad Checkpoint	$O(VL)$ activations	+33% compute	None	Memory-constrained runs	PyTorch built-in

In practice: always enable Mixed Precision + ZeRO-1. Add Gradient Checkpointing if still OOM. Scale up with 3D Parallelism for 70B+.

§5

Implementation

PyTorch DDP · FSDP · DeepSpeed · Benchmarks

PyTorch DistributedDataParallel (DDP)



Minimal code changes to go from 1 GPU to N GPUs

```
import torch, torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP

# 1. Initialize process group
dist.init_process_group(backend='nccl')
local_rank = int(os.environ['LOCAL_RANK'])
torch.cuda.set_device(local_rank)

# 2. Wrap model with DDP
model = MyModel().cuda(local_rank)
model = DDP(model, device_ids=[local_rank])

# 3. Distributed sampler — different batch each GPU
sampler = DistributedSampler(dataset)
loader = DataLoader(dataset, sampler=sampler)

# 4. Training loop — unchanged!
for batch in loader:
    loss = criterion(model(batch), labels)
    loss.backward() # DDP syncs grads automatically
    optimizer.step(); optimizer.zero_grad()
```

```
torchrun --nproc_per_node=8 train.py
```



Single-process-per-GPU model — one rank per GPU



AllReduce happens automatically during backward()



Synchronises parameters at init — consistent start



DistributedSampler ensures no data overlap between ranks



Limitation: full model on every GPU — OOM for 70B+



PyTorch FSDP — Full Sharding

ZeRO-3 equivalent — shards everything across GPUs

```
from torch.distributed.fsdp import FullyShardedDataParallel as FSDP
from torch.distributed.fsdp import MixedPrecision
from torch.distributed.fsdp.wrap import transformer_auto_wrap_policy

# Auto-wrap each transformer block independently
auto_wrap = functools.partial(
    transformer_auto_wrap_policy,
    transformer_layer_cls={TransformerBlock})

bf16_policy = MixedPrecision(param_dtype=torch.bfloat16,
                             reduce_dtype=torch.bfloat16, buffer_dtype=torch.bfloat16)

model = FSDP(model, # wraps each TransformerBlock
             auto_wrap_policy=auto_wrap,
             mixed_precision=bf16_policy,
             sharding_strategy=ShardingStrategy.FULL_SHARD)
```

```
# DeepSpeed equivalent — ds_config.json

"zero_optimization": {"stage": 3, "offload_optimizer": {"device": "cpu"}}

deepspeed --num_gpus=8 train.py --deepspeed ds_config.json
```

DDP vs FSDP

DDP: Full model on every GPU
FSDP: Shards params across all GPUs

Memory (FSDP): $\text{model_size} / N$

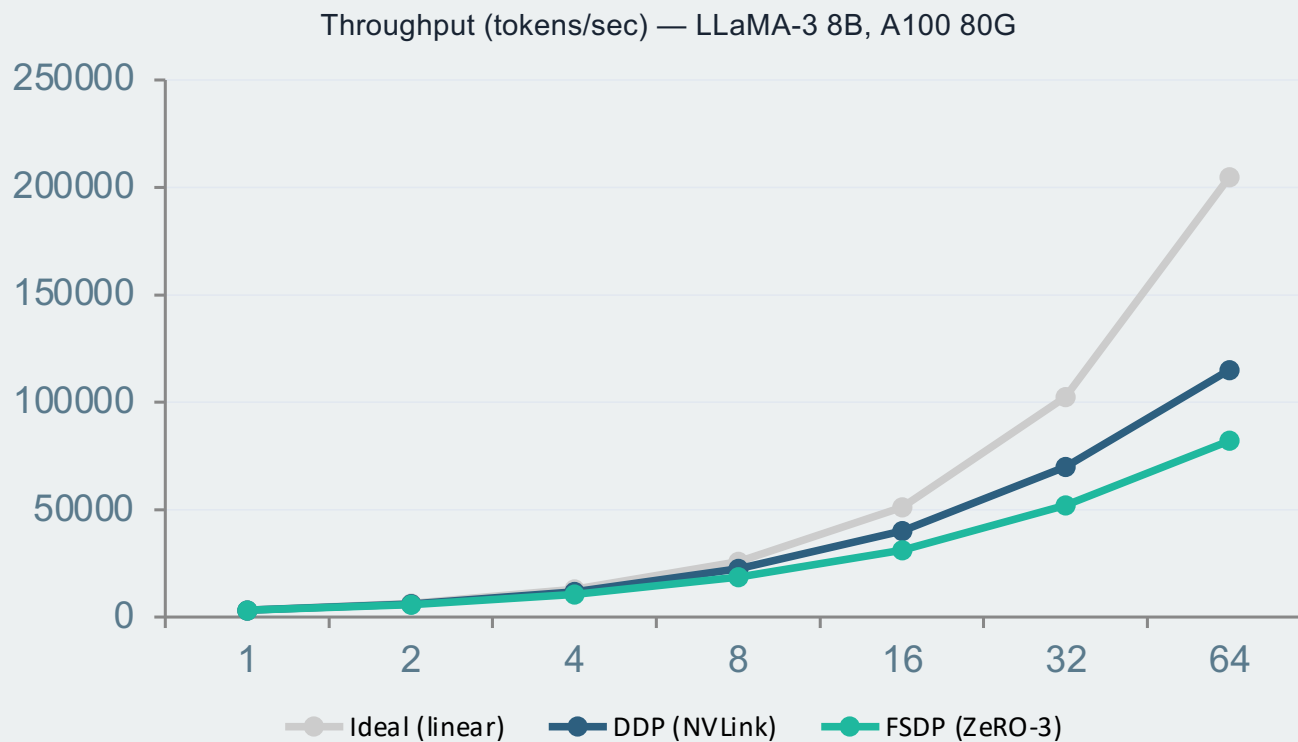
FSDP needs:

- AllGather before forward()
- ReduceScatter after backward()

Use when DDP OOMs on your model.

Throughput Scaling — Real-World Numbers

Tokens/second vs. GPU count for LLaMA-3 8B (A100 80G)



~90%

DDP efficiency at 2 GPUs
(NVLink)

~70%

DDP efficiency at 32 GPUs
(multi-node)

~55%

FSDP efficiency at 32 GPUs
(extra comms)

~40%

DDP efficiency at 64 GPUs
(comm saturates)

Common Pitfalls & Debugging



The errors you will hit on your first distributed training run

Issue	Root Cause	Fix	Sev.
NCCL timeout / hang	Process group init fails — one rank never joins	Check firewall, ensure all ranks call <code>init_process_group()</code>	●
OOM on rank 0 only	Model loaded on CPU then moved to GPU 0 before DDP wrap	Move model to device BEFORE wrapping with DDP/FSDP	●
Loss diverges vs 1-GPU	Learning rate not scaled with effective batch size	Scale LR by $\sqrt{N}$ or N (linear scaling rule)	●
Uneven batch sizes	Dataset not divisible by <code>world_size</code>	Use <code>DistributedSampler(drop_last=True)</code>	●
FSDP save/load crash	FSDP wraps each layer — can't save as normal <code>state_dict</code>	Use <code>FSDP.state_dict()</code> with <code>FullStateDictConfig</code>	●
Pipeline bubble too large	Too few microbatches relative to stages	Increase microbatch count: $m \geq 4 \times p$	●
Stale gradient warning	Unused parameters in model with DDP	Set <code>find_unused_parameters=True</code> in DDP wrapper	●

KEY TAKEAWAYS —Module 10

Data parallelism is the default starting point Same model on every GPU, different data shards. AllReduce syncs gradients. Works up to ~70B with ZeRO.

Ring-allreduce is bandwidth-optimal Communication cost = $2(N-1)/N \times |\text{grad}|$. Near-optimal for large N. NCCL handles this for you.

Model parallelism is for layers that won't fit Tensor parallelism splits within a layer (NVLink required). Pipeline parallelism splits across layers (multi-node).

Pipeline bubble is the key overhead Bubble fraction = $(p-1)/(m+p-1)$. Use $m \geq 4p$ microbatches to keep idle time below 20%.

ZeRO-3 / FSDP enables 70B+ on commodity GPUs Full parameter sharding: memory per GPU = total model / N. Extra AllGather/ReduceScatter cost.

Always combine strategies in practice Mixed Precision (free win) + ZeRO-1 (free win) + Gradient Checkpointing if OOM. 3D parallelism for frontier scale.